Báo cáo kết quả huấn luyện & Thử nghiệm

Đề tài: Phát hiện trùng lặp mã nguồn C/C++ và Python

Bộ test mở rộng: danh sách liên kết + hướng đối tượng (theo góp ý của thầy)

Ngày 17 tháng 10 năm 2026

1 Kết quả tuần này

- Thầy góp ý bộ test còn ngắn: cần thêm code dài, có hàm, cấu trúc danh sách liên kết, hướng đối tượng, và phần `struct` với `class` gần như hoàn toàn giống nhau.
- Thêm nhóm **P5** vào bộ test `custom_suite`: 6 file, mỗi file 59–71 dòng (các bài P1–P4 chỉ 11–17 dòng), dùng `struct Node` + 4 hàm tự do hoặc `class IntList` + 4 phương thức trên cùng một thuật toán.
- Chấm 8 cặp mới (bộ test từ 26 lên 34 cặp) bằng 6 phương pháp: MOSS server thật (B1) và M1 CodeBERT, M2 GraphCodeBERT, M3 ASTNN, M4 CPG-GNN, M5 Hybrid. M1/M2/M5 dùng bản 512 token, ngưỡng đọc từ `threshold.json` của từng run.
- Chuyển `data/custom_suite` vào git (trước đây cả thư mục `data/` bị ignore) để code test nằm trong repo; hình minh hoạ sinh lại được bằng `scripts/evidence_custom_suite.py`.
- Kết quả: **cả 5 model bắt đúng** cặp `struct` ↔ `class` (M1/M2/M5 ≈ 1,000; M3 0,915; M4 0,973) — **MOSS bỏ sót cả hai** cặp clone dạng `class` (0,000). M3 đúng cả 8 cặp; M4 sai 3 (toàn dương giả).
- Hai điều mới lộ ra: (a) cặp cross `X_P1P5` (DSLK vs tổng $1..n$) bị M1/M2/M5 nói sai ≈ 1,000 vì hai bài cùng phép toán tổng; (b) cả 6 file P5 đều vượt 512 subword token nên M1/M2/M5 phải cắt tới 25% code — chi tiết ở §3.

2 Nhóm test P5: danh sách liên kết + hướng đối tượng

2.1 Sáu file và tám cặp

- `p5_base_list.cpp` (61 dòng): `struct Node` + 4 hàm tự do `pushBack`, `sumList`, `printList`, `freeList`; đọc n số, in dãy và in tổng.
- `p5_v_class.cpp` (71 dòng): **cùng hành vi**, nhưng viết bằng `class IntList` + 4 phương thức `pushBack`, `sum`, `print`, `clear`, con trỏ `head` để trong đối tượng — đây là cặp thầy muốn xem.
- `p5_v_while.cpp` (67), `p5_v_scanf.cpp` (60): biến thể clone dễ (đổi vòng lặp, đổi IO, đổi tên biến).
- `p5_v_combo.cpp` (68): biến thể clone khó (while + scanf + đảo điều kiện + biến thừa).
- `p5_n_palindrome.cpp` (69): cùng khung `class` + `Node` nhưng **khác ngữ nghĩa** — kiểm tra dãy đối xứng (in YES/NO) thay vì tính tổng, dùng để thử xem model có bị khung code giống nhau đánh lừa không.

pair_id	File A / File B	GT	Kiểu
P5_struct_class	base / v_class	clone	struct + hàm → class + phương thức
P5_while	base / v_while	clone	for → while + đổi tên biến
P5_scanf	base / v_scanf	clone	cin/cout → scanf/printf + đổi tên
P5_combo	base / v_combo	clone	while + scanf + đảo điều kiện + biến thừa
P5_class_combo	v_class / v_combo	clone	hai dạng biến đổi cùng lúc
P5_nonclone_palindrome	base / n_palindrome	non-clone	cùng khung Node/class, khác ngữ nghĩa
X_P1P5	base / p1_base_sum	non-clone	cross-problem (DSLK với tổng $1..n$)
X_P2P5	base / p2_base_gcd	non-clone	cross-problem (DSLK với UCLN)

2.2 Code: bản `struct` và bản `class`

Hai bản giống nhau gần như từng câu lệnh; khác ở chỗ bản A để `Node` và các hàm ở ngoài (kiểu C), bản B gói `Node` vào trong `class IntList` và biến hàm thành phương thức, con trỏ `head` thành thuộc tính riêng.

File A — `p5_base_list.cpp` (struct + hàm)

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node *next;
};

Node* pushBack(Node *head, int v) {
    Node *node = new Node;
    node->val = v;
    node->next = NULL;
    if (head == NULL) {
        return node;
    }
    Node *p = head;
    while (p->next != NULL) {
        p = p->next;
    }
    p->next = node;
    return head;
}

long long sumList(Node *head) {
    long long total = 0;
    for (Node *p = head; p; p = p->next) {
        total = total + p->val;
    }
    return total;
}
```

File B — `p5_v_class.cpp` (class + phương thức)

```
#include <bits/stdc++.h>
using namespace std;

class IntList {
public:
    struct Node {
        int val;
        Node *next;
    };

    IntList() {
        head = NULL;
    }

    void pushBack(int v) {
        Node *node = new Node;
        node->val = v;
        node->next = NULL;
        if (head == NULL) {
            head = node;
            return;
        }
        Node *p = head;
        while (p->next != NULL) {
            p = p->next;
        }
        p->next = node;
    }

    long long sum() {
        long long total = 0;
        for (Node *p = head; p; p = p->next) {
            total = total + p->val;
        }
        return total;
    }
```

Trích phần lõi: mỗi thân hàm ở bản B lặp lại gần từng câu lệnh của bản A, chỉ thêm một tầng thụt lề và bỏ tham số `head` (con trỏ `head` nằm trong đối tượng). Hai file còn dài hơn (61 và 71 dòng với `print`/`clear` và `main`); xem đủ ở Hình 4.1 và trong `data/custom_suite/`.

3 Kết quả chấm 8 cặp mới

Cặp	GT	B1 MOSS	M1	M2	M3	M4	M5
P5_struct_class	clone	0,000	1,000	1,000	0,915	0,973	1,000
P5_while	clone	0,540	1,000	1,000	0,993	0,973	1,000
P5_scanf	clone	0,710	1,000	1,000	0,995	0,973	1,000
P5_combo	clone	0,150	1,000	1,000	0,994	0,973	1,000
P5_class_combo	clone	0,000	1,000	1,000	0,907	0,973	1,000
P5_nonclone_palindrome	non-clone	0,000	0,002	0,000	0,270	0,743	0,002
X_P1P5	non-clone	0,000	1,000	1,000	0,179	0,742	1,000
X_P2P5	non-clone	0,000	0,000	0,000	0,375	0,710	0,000
Ngưỡng (theo run)		0,010	0,210	0,660	0,420	0,470	0,010
Số cặp sai / 8		2	1	1	**0**	3	1

Số là $P(\text{clone})$ từng phương pháp trả về (0–1). Đỏ = nói sai (so với ngưỡng của chính run). Ví dụ: M3 ở `P5_struct_class` được 0,915 > ngưỡng 0,420 nên nói clone (đúng).

Cặp `struct` ↔ `class`: cả 5 model đều đúng, MOSS bỏ sót

Cặp thầy quan tâm (`P5_struct_class` và `P5_class_combo`) đều được cả 5 model nhận là clone: M1/M2/M5 $\approx$ 1,000; M3 0,915 và 0,907; M4 0,973 và 0,973. Tức việc bọc `Node` + hàm tự do vào `class` + phương thức **không** làm model mất dấu. Ngược lại MOSS **bỏ sót cả hai** (0,000) — đúng hành vi đã thấy ở bộ test cũ: server chỉ trả về cặp có chùm fingerprint dài, còn biến đổi cấu trúc nhiều (Type-3/4) thì không được gắn cờ. 3 cặp clone còn lại cả 5 model đều đúng ($\geq$ 0,907), MOSS bắt được 3/5.

Non-clone khó: M4 bị khung code đánh lừa, M3 thì không

`P5_nonclone_palindrome` dùng đúng khung `class` + `Node` nhưng khác ngữ nghĩa (kiểm tra dãy đối xứng thay vì tính tổng). Lần này chỉ **M4** nói sai (0,743 $>$ 0,470); M1 0,002, M2 0,000, M5 0,002 và M3 0,270 đều đúng. Khác bộ test cũ (nơi M3 và M4 cùng bị khung code lừa 10 cặp), ở nhóm P5 **M3 đúng cả 8 cặp** và M4 là model sai nhiều nhất.

Lỗi mới ở cặp cross: `X_P1P5`

`X_P1P5` (danh sách liên kết vs tính tổng $1..n$) bị M1/M2/M5 cho $\approx$ 1,000 và M4 cho 0,742 — **nói sai là clone**, chỉ M3 đúng (0,179). Đây là lỗi mới: ở 6 cặp cross cũ (sum vs gcd/max/prime) M1/M2/M5 đều $\leq$ 0,174. Đọc kỹ thì hai bài này *cùng phép toán tổng* và cùng khung `main` (đọc n, vòng `for`, `cout`, `return`), nên cặp khó hơn hẳn về ngữ nghĩa chứ không chỉ về hình thức; MOSS vẫn đúng (0,000) vì không thấy chùm fingerprint chung đủ dài. `X_P2P5` (DSLK vs UCLN) thì chỉ M4 sai.

Giới hạn độ dài đầu vào vừa lộ ra

Cả 6 file P5 đều **vượt 512 subword token** (520 ở `p5_v_scanf` tới 683 ở `p5_v_class`), nên M1/M2/M5 (chạy `max_length` 512) chỉ đọc được phần đầu: cắt 1,7% ở `p5_base_list` (521 token) và tới **25%** ở `p5_v_class` (683 token). M3 (cây AST) và M4 (node CPG) không bị cắt theo token: CPG của 6 file P5 chỉ 61–95 node, xa trần 1500 node của M4. Nghĩa là cặp `P5_struct_class` được quyết định khi bản `class` hụt 1/4 số token cuối (trong đó có cả `main`) mà cả 5 model vẫn đúng — nhưng nếu code dài hơn nữa thì đây sẽ là điểm nghẽn thật, cần nới `max_length` hoặc cắt theo hàm (việc đã hoãn từ DEC-029). Tổng trên cả 34 cặp: M3 10, M4 13, B1 18, M1 3, M2 3, M5 4 lỗi.

4 Minh hoạ

Cách đọc hình:

- Hai phía cùng màu là hai đoạn mà model cho là khớp nhau.
- Mỗi model có một bảng màu riêng.
- CodeBERT, GraphCodeBERT, Hybrid ghép theo từng từ (vector encoder).
- ASTNN ghép theo câu lệnh (vector RvNN); bỏ các span chiếm hơn 60% file và chỉ giữ statement cùng cấp, không bôi cả hàm.
- CPG-GNN ghép theo dòng (vector node GAT).
- MOSS tô đúng vùng khớp mà server Stanford trả về; cặp nào không được gắn cờ thì để trắng.
- Ô vàng lẻ là token model dựa vào để cho điểm nhưng không ghép được cặp, phần không tô màu là còn lại.

4.1 Cặp clone `struct` ↔ `class`: P5_struct_class (M2)

Bản A (trái) viết `struct Node` + hàm tự do; bản B (phải) gói `Node` vào `class IntList` và biến hàm thành phương thức. M2 cho 1,000 (ngưỡng 0,660) — nói đúng là clone. Hai phía cùng màu là những chỗ M2 ghép hai bản với nhau: gần như toàn bộ thân `pushBack`/`sum`/`print` trùng nhau từng câu lệnh.

GraphCodeBERT — 1,000 (clone)

File A (p5_base_list)

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
int val;
Node *next;
};

Node* pushBack(Node *head, int v) {
Node *node = new Node;
node->val = v;
node->next = NULL;
if (head == NULL) {
return node;
}
Node *p = head;
while (p->next != NULL) {
p = p->next;
}
p->next = node;
return head;
}

long long sumList(Node *head) {
long long total = 0;
for (Node *p = head; p; p = p->next) {
total = total + p->val;
}
return total;
}

void printList(Node *head) {
for (Node *p = head; p; p = p->next) {
cout << p->val << " ";
}
cout << endl;
}

void freeList(Node *head) {
while (head != NULL) {
Node *nxt = head->next;
delete head;
head = nxt;
}
}

int main() {
int n;
cin >> n;
Node *head = NULL;
for (int i = 0; i < n; i++) {
int x;
cin >> x;
head = pushBack(head, x);
}
printList(head);
cout << sumList(head) << endl;
freeList(head);
return 0;
}
```

File B (p5_v_class)

```
#include <bits/stdc++.h>
using namespace std;

class IntList {
public:
struct Node {
int val;
Node *next;
};

IntList() {
head = NULL;
}

void pushBack(int v) {
Node *node = new Node;
node->val = v;
node->next = NULL;
if (head == NULL) {
head = node;
return;
}
Node *p = head;
while (p->next != NULL) {
p = p->next;
}
p->next = node;
}

long long sum() {
long long total = 0;
for (Node *p = head; p; p = p->next) {
total = total + p->val;
}
return total;
}

void print() {
for (Node *p = head; p; p = p->next) {
cout << p->val << " ";
}
cout << endl;
}

void clear() {
while (head != NULL) {
Node *nxt = head->next;
delete head;
head = nxt;
}
}

private:
Node *head;
};

int main() {
int n;
cin >> n;
IntList list;
for (int i = 0; i < n; i++) {
int x;
cin >> x;
list.pushBack(x);
}
list.print();
cout << list.sum() << endl;
list.clear();
return 0;
}
```

4.2 Cặp cross bị nói sai: X_P1P5 (M2)

Hai bài đều là code C++ dài đọc n rồi in một số, nhưng một bài cộng các phần tử của danh sách liên kết còn bài kia cộng $1..n$: *khác đề* nhưng cùng phép toán tổng và cùng khung `main`. M2 cho 0,999967 (ngưỡng 0,660) nên kết luận sai là clone; hình cho thấy chỗ ghép chủ yếu nằm ở khung chung (`main`, `cin`/`cout`, vòng `for`, tên biến `n`, `x`), không phải ở phần xử lý riêng mỗi bài.

GraphCodeBERT — 1,000 (clone)

File A (p5_base_list)

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
int val;
Node *next;
};

Node* pushBack(Node *head, int v) {
Node *node = new Node;
node->val = v;
node->next = NULL;
if (head == NULL) {
return node;
}
Node *p = head;
while (p->next != NULL) {
p = p->next;
}
p->next = node;
return head;
}

long long sumList(Node *head) {
long long total = 0;
for (Node *p = head; p; p = p->next) {
total = total + p->val;
}
return total;
}

void printList(Node *head) {
for (Node *p = head; p; p = p->next) {
cout << p->val << " ";
}
cout << endl;
}

void freeList(Node *head) {
while (head != NULL) {
Node *nxt = head->next;
delete head;
head = nxt;
}
}

int main() {
int n;
cin >> n;
Node *head = NULL;
for (int i = 0; i < n; i++) {
int x;
cin >> x;
head = pushBack(head, x);
}
printList(head);
cout << sumList(head) << endl;
freeList(head);
return 0;
}
```

File B (p1_base_sum)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
int n, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) {
sum += i;
}
cout << sum << endl;
return 0;
}
```

5 Hạn chế

6 Kế hoạch tiếp theo